



# Hybrid Post-Quantum Retrofit: Upgrading a Live Pro- tocol Without a Fork

---

Zach Kelling

*Hanzo AI*

2026

## Abstract

A compute market protects its users' funds with signatures and its users' work with encryption, and both rest on elliptic curves. Neither survives a cryptographically relevant quantum computer. The usual objection to acting now is that migration means a fork, a flag day, and a coordination problem across an open provider network.

It does not have to. We describe a retrofit pattern that adds post-quantum identity and key exchange to a running protocol without changing its wire format or forking the reference implementation, and we report two independent deployments of it: one onto a mesh networking stack with a 500-byte maximum transmission unit and a 5 bit-per-second floor, and one onto a defence-grade file storage and transfer profile. The first is the harder constraint by a wide margin, and it fits.

We then apply the pattern surface by surface to a decentralized compute market — provider identity, session establishment, job receipts, staking signatures, governance — give the byte cost of each, and propose a migration order derived from key lifetime rather than from implementation convenience.

## Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Two clocks, not one</b>                         | <b>3</b> |
| <b>2</b> | <b>The pattern</b>                                 | <b>3</b> |
| <b>3</b> | <b>Two deployments</b>                             | <b>4</b> |
| 3.1      | Mesh networking under extreme constraint . . . . . | 4        |
| 3.2      | Defence-grade storage and transfer . . . . .       | 4        |
| <b>4</b> | <b>Policy as a separate object</b>                 | <b>4</b> |
| <b>5</b> | <b>Applying it to a compute market</b>             | <b>4</b> |
| 5.1      | Byte budget . . . . .                              | 4        |
| 5.2      | A conservative root of trust . . . . .             | 5        |
| <b>6</b> | <b>What adoption looks like</b>                    | <b>5</b> |

## 1 Two clocks, not one

Quantum risk is usually discussed as a single future event. For protocol design it is two separate deadlines with different urgencies.

**Confidentiality has already expired.** An adversary recording session transcripts today can decrypt them whenever a capable machine exists. Every session established with a classical key exchange is retroactively readable. The deadline for key exchange was whenever recording began.

**Authenticity expires later, but the key does not rotate.** A signature forged after the fact is worthless against a transcript already accepted, so signing is less urgent — unless the key is long-lived, in which case an adversary who recovers it can impersonate the holder from that moment forward. Provider identity keys, staking keys and governance keys are exactly that: keys whose value is their continuity.

The ordering falls out. Key exchange first because the loss is already accruing; long-lived identity next because it cannot be rotated cheaply; ephemeral per-job signatures last because they can simply be reissued.

## 2 The pattern

The retrofit is additive, hybrid, and wire-compatible.

**Additive.** The post-quantum material is carried alongside the classical material, not instead of it. An old peer ignores a field it does not recognise; a new peer verifies both. No version negotiation, no flag day.

**Hybrid.** Security is the maximum of the two assumptions, not the minimum. The session key derives from both shared secrets through a domain-separated key derivation, so the construction is at least as strong as the stronger component. A break in the lattice assumption degrades to classical security; a quantum adversary degrades to lattice security. This matters because lattice cryptography is young, and a market should not bet its custody on the newest assumption alone.

**Wire-compatible.** The transport, framing, session state machine and authentication flow are untouched. This is the property that makes it a retrofit rather than a migration.

| Layer                    | Classical         | Hybrid replacement                               |
|--------------------------|-------------------|--------------------------------------------------|
| Identity signing         | Ed25519           | Ed25519 + ML-DSA-65 (detached pair, both verify) |
| Key exchange             | X25519            | X-Wing = X25519 + ML-KEM-768                     |
| Key exchange, high value | X448              | X448 + ML-KEM-1024                               |
| Bulk AEAD                | AES-256-GCM       | unchanged                                        |
| Bulk AEAD, software      | ChaCha20-Poly1305 | unchanged                                        |
| Hash                     | SHA-256           | SHA-384 / SHA3-384                               |
| Derivation               | HKDF              | HKDF-SHA-384, domain-separated per layer         |

Table 1: The substitution table. Symmetric primitives are left alone: Grover halves the effective key length, and 256-bit keys are already sized for that.

## 3 Two deployments

### 3.1 Mesh networking under extreme constraint

The first deployment extends a cryptography-based mesh networking stack designed for LoRa, packet radio, KISS modems, serial lines and free-space optical links — any half-duplex carrier with at least 5 bits per second and a 500-byte maximum transmission unit. It provides initiator-anonymous, coordination-free multi-hop transport entirely without IP.

Adding a 1952-byte public key and a 3309-byte signature to a protocol with a 500-byte frame is the least forgiving environment we could have chosen, which is why it was the right test. Identity signing moved to Ed25519 with ML-DSA-65 alongside; key exchange to X25519 with ML-KEM-768; session encryption and derivation unchanged, now over the hybrid secret. The reference protocol was not forked.

### 3.2 Defence-grade storage and transfer

The second is an envelope-encryption profile for file storage and transfer that encrypts before content enters any storage or transfer system, so the back end sees only ciphertext and opaque manifests — sender, recipients, filenames and classification labels never leave the client. Bulk encryption is AES-256-GCM in the certified profile or ChaCha20-Poly1305 in software, hashing is SHA-384 or SHA3-384, key wrapping is X-Wing for the general case and X448 with ML-KEM-1024 for high-value material, and the sender signature is a detached Ed25519 and ML-DSA-65 pair.

The two deployments share no code and no threat model. What they share is the pattern, which is the point.

## 4 Policy as a separate object

A hybrid deployment needs a way to say “classical is no longer acceptable here” without that statement being scattered across every call site. We keep it as one value and one gate.

The policy is a profile carrying one prohibition flag per cryptographic primitive family, with a zero value meaning classical semantics and a strict constructor setting every flag. Operations report themselves through a small fixed set of stable identifiers covering the precompile families and polynomial commitments. Every constrained operation calls a single refusal function once at the top of its entry point and otherwise proceeds in its own lane.

The property worth copying is the decompilation: the profile does not know how operations compute, and operations do not know how the profile was selected. Verification logic and policy enforcement are not braided together, so either can be replaced without touching the other.

A second, duller lesson is worth stating because it cost us real bugs. The profile and policy identifier strings were independently reimplemented in Go, in Solidity, in TypeScript and in a paper, and they drifted — one surface used a name the on-chain state did not recognise. Identifiers of this kind should have exactly one source of truth, generated into every consuming language. We now publish them as a single package with Go, Solidity and TypeScript subpackages sharing one set of strings.

## 5 Applying it to a compute market

### 5.1 Byte budget

Adoption is a bandwidth question, and the numbers are modest at market scale.

A signed job receipt grows by roughly 3.3 kB. At a million jobs a day that is 3.3 GB of additional signature traffic — entirely unremarkable for a system already moving model weights.

| Surface                | Key lifetime | Exposure                   | Priority |
|------------------------|--------------|----------------------------|----------|
| Session establishment  | seconds      | harvest-now, decrypt-later | first    |
| Provider identity      | years        | impersonation on recovery  | first    |
| Staking / custody keys | years        | direct theft               | first    |
| Governance keys        | years        | protocol capture           | second   |
| Job receipts           | minutes      | forgery after acceptance   | third    |
| Client API tokens      | months       | rotate instead             | rotate   |

Table 2: Migration order derived from key lifetime and loss on compromise.

| Object                  | Classical | Hybrid  |
|-------------------------|-----------|---------|
| Identity public key     | 32 B      | 1,984 B |
| Identity signature      | 64 B      | 3,373 B |
| Key-exchange public key | 32 B      | 1,216 B |
| Key-exchange ciphertext | 32 B      | 1,120 B |

Table 3: Wire cost of the hybrid substitution. ML-DSA-65 public keys are 1,952 B and signatures 3,309 B; ML-KEM-768 public keys are 1,184 B and ciphertexts 1,088 B; classical components add 32 B each.

Session establishment grows by about 2.3 kB once per session, amortised across every request in it.

## 5.2 A conservative root of trust

Lattice assumptions are the right default for volume: fast, small, standardised. For the single most valuable key in the system — the one that anchors governance or custody — there is an argument for a different hardness assumption entirely. Hash-based signatures under FIPS 205 rest only on the preimage resistance of a hash function, an assumption with a far longer track record and no algebraic structure for a future attack to exploit. They are larger and slower, which is irrelevant for a key used a handful of times a year.

Our threshold signing daemon carries this as a first-class option alongside the lattice families, and also supports composing two orthogonal lattice families so that both must verify — a hedge against a break in one lattice problem rather than a break in lattices generally.

## 6 What adoption looks like

The retrofit does not require adopting our transport, our consensus or our tooling. It requires an implementation of ML-DSA-65 and ML-KEM-768, a domain-separated derivation, an additive wire field, and a dual-verify window during which both signatures are checked and only the classical one is required. The window closes when the population has migrated, at which point the classical field becomes optional and eventually ignored.

Our implementations of the primitives, the hybrid envelope profile, the mesh retrofit, the policy gate and the shared identifier package are open source under permissive licence, with known-answer test vectors. They are usable independently of each other and of everything else we build.